

Pemrograman Berorientasi Object

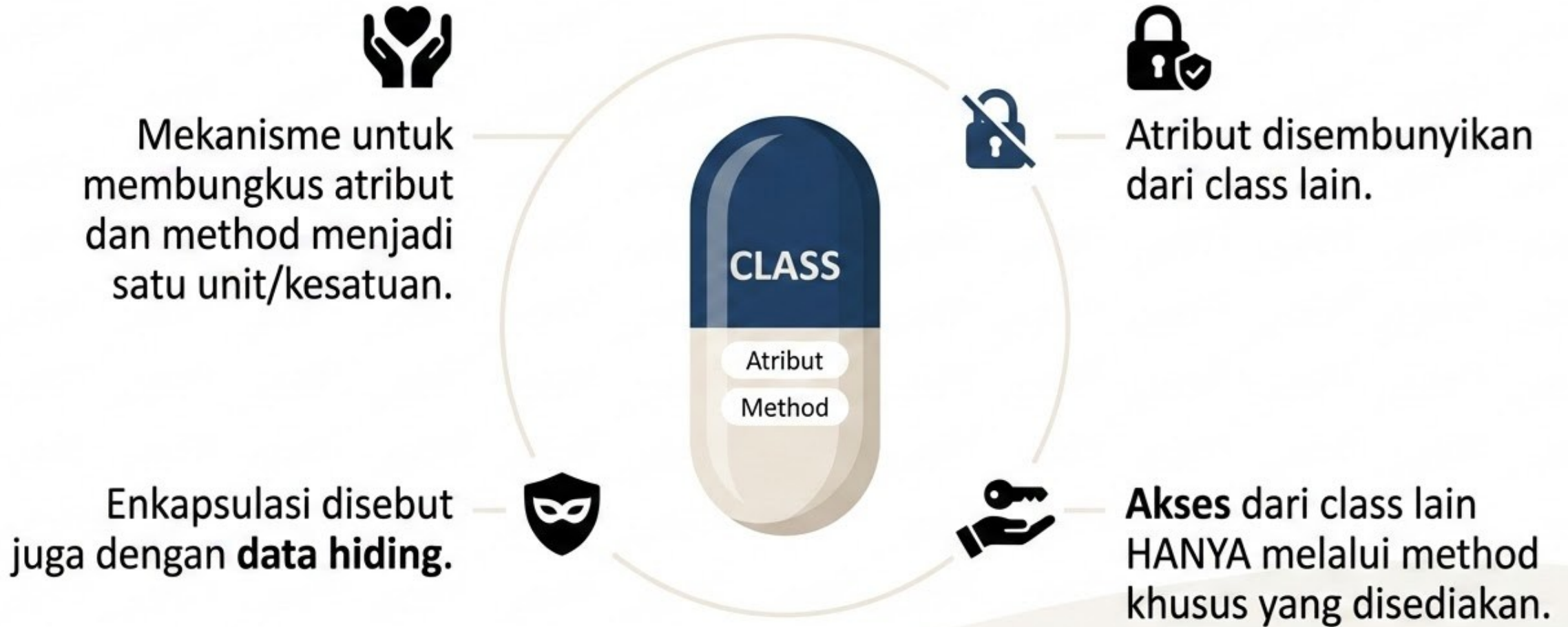
Enkapsulasi

Tim PBO

Outline

- Access Modifier
 - Getter – Setter
 - Konstruktor
 - Notasi UML
- 

Enkapsulasi



Menganalisa Skenario

Mari kita amati behaviour dari objek di dunia nyata.

- Sebagai contoh, sebuah mobil.
- Bagaimana perilaku dari sebuah mobil?
 - Nyalakan mesin. Tambah kecepatan. Tambah kecepatan lagi. Kurangi kecepatan. Berhenti. Mundur pelan. Berhenti.
- Behaviour tersebut dapat dilaksanakan melalui beberapa mekanisme.
 - Kunci kontak untuk menyalakan dan mematikan mesin. Tongkat persneling, pedal gas dan pedal rem untuk menambah atau mengurangi kecepatan.
- Dalam hal ini, kita menggunakan pedal gas dan pedal rem untuk mengatur kecepatan mobil tersebut.

Menganalisa Skenario

```
public class Mobil
{
    public int kecepatan = 0;
    public boolean kontakOn = false;

    public void tampilkanStatus()
    {
        if(kontakOn == true)
        {
            System.out.println("Kontak ON");
        }
        else
        {
            System.out.println("Kontak OFF");
        }

        System.out.println("Kecepatan: " + kecepatan + "\n");
    }
}

public class TestMobil
{
    public static void main(String[] args)
    {
        Mobil mbl = new Mobil();

        mbl.tampilkanStatus();
        mbl.kecepatan = 100;
        mbl.tampilkanStatus();
    }
}
```

```
E:\Java>javac TestMobil.java
```

```
E:\Java>java TestMobil
```

```
Kontak OFF
Kecepatan: 0
```

```
Kontak OFF
Kecepatan: 100
```

Menganalisa Skenario

Ada yang janggal dari class Mobil.

- Yaitu, kecepatan mobil tiba-tiba saja berubah dari 0 ke 100.
- Lebih janggal lagi, posisi kontak mobil masih dalam kondisi OFF.
- Bagaimana mungkin sebuah mobil bisa sekejap berkecepatan dari nol ke 100, dan itupun kunci kontaknya OFF?
- Nah dalam hal ini, akses ke atribut Mobil ternyata tidak terkontrol.
- Padahal, objek di dunia nyata selalu memiliki batasan dan mekanisme bagaimana objek tersebut dapat digunakan.

Solusi

- Menyembunyikan atribut internal (kecepatan, kontakOn) dari pengguna (class lain)
 - Menyediakan *method* khusus untuk mengakses atribut.
- 

Access Modifier

- Java menyediakan 4 akses modifier yang berbeda untuk mengontrol akses terhadap atribut maupun method dari class lain, yaitu:

 *private* – paling restriktif, hanya bisa diakses di dalam class itu sendiri.

 *default* – hanya dapat diakses di dalam package yang sama.

 *protected* – dapat diakses di dalam package dan sub class.

 *public* – paling tidak restriktif, dapat diakses dari mana saja.

Aksesibilitas Modifier

	Same Class Same Package	Other Class Same Package	SubClass Other Package	Other Class Other Package
public	✓	✓	✓	✓
protected	✓	✓	✓	
default	✓	✓		
public	✓			

Default Access Modifier (Package-Private)

Karakteristik: Modifier ini tidak memerlukan kata kunci (*keyword*) apa pun saat dideklarasikan. Atribut atau method yang menggunakan tingkat akses ini hanya dapat diakses oleh class-class lain yang berada di dalam **package** yang sama.

1

Contoh Kasus: Melanjutkan skenario sistem perbankan, misalkan kita memiliki kelas internal bernama **KaryawanBank** yang bertugas memproses data nasabah dalam package yang sama (**perbankan_internal**), namun kita tidak ingin data tersebut diakses oleh package luar.

```
5 package perbankanInternal;
6
7 /**
8  *
9  * @author dian
10  */
11 public class KaryawanBank {
12     // Menggunakan default modifier (tanpa keyword)
13     String idKaryawan;
14     String departemen;
15
16     // Default constructor
17     KaryawanBank(String id, String dept) {
18         this.idKaryawan = id;
19         this.departemen = dept;
20     }
21
22     // Default method
23     void prosesTransaksi() {
24         System.out.println("Karyawan " + idKaryawan + " "
25                             + " sedang memproses transaksi.");
26     }
27 }
```

Protected Access Modifier

Karakteristik: Atribut atau method yang dideklarasikan dengan keyword **protected** dapat diakses oleh class lain di dalam **package yang sama**, serta oleh **sub-class (class turunan)** meskipun sub-class tersebut berada di package yang berbeda.

Contoh Kasus: Masih dalam sistem perbankan, sebuah akun bank umum (**AkunBank**) memiliki mekanisme dasar pengelolaan. Kita ingin membuat sub-class khusus seperti **AkunGiro** atau **AkunPremium** yang dapat mewarisi dan memodifikasi beberapa parameter internal tanpa membukanya secara bebas (**public**) ke objek luar.

```
5 package layananUtama;
6
7 /**
8  *
9  * @author dian
10  */
11 public class AkunBank {
12     // Menggunakan protected modifier
13     protected String nomorAkun;
14     protected double tingkatBunga;
15
16     public AkunBank(String nomorAkun, double tingkatBunga) {
17         this.nomorAkun = nomorAkun;
18         this.tingkatBunga = tingkatBunga;
19     }
20
21     // Protected method untuk penyesuaian internal
22     protected void sesuaikanBunga(double bungaBaru) {
23         this.tingkatBunga = bungaBaru;
24         System.out.println("Tingkat bunga untuk akun " + nomorAkun +
25                             " disesuaikan menjadi: " + tingkatBunga + "%");
26     }
27 }
```

Penerapan pada Sub-Class (Package Berbeda):

```
5 package layananKhusus;
6 import layananUtama.AkunBank;
7
8
9 /**
10  *
11  * @author dian
12  */
13 public class AkunPremium extends AkunBank {
14     private String benefitKhusus;
15
16     public AkunPremium(String nomorAkun, double tingkatBunga, String benefit) {
17         // Memanggil konstruktor super-class
18         super(nomorAkun, tingkatBunga);
19         this.benefitKhusus = benefit;
20     }
21
22     public void berikanBonusBunga() {
23         // Atribut 'tingkatBunga' dari AkunBank dapat diakses langsung karena
24         // bersifat protected
25         double bungaBonus = tingkatBunga + 1.5;
26
27         // Method 'sesuaikanBunga' dapat dipanggil langsung oleh class turunan
28         sesuaikanBunga(bungaBonus);
29         System.out.println("Benefit tambahan: " + benefitKhusus);
30     }
31 }
```

Notasi UML Access Modifier

- Notasi UML pada penggambaran class diagram untuk access modifier adalah:
 - Tanda minus (-) untuk *private*
 - Tanda pagar (#) untuk *protected*
 - Tanda plus (+) untuk *public*
 - Tanda tilde (~) atau tanpa tanda untuk *default*

SpaceShuttle
+ color: String - speed: int - orbitHeight: int # fuelType: String # engineType: String nickName: String
+ start(): void + climb(): void + dive(): void + getSpeed(): int + getOrbitHeight(): int

Menerapkan Access Modifier

Mobil
- kecepatan: int - kontakOn: boolean
+ nyalakanMesin(): void + matikanMesin(): void + tambahKecepatan(): void + kurangiKecepatan(): void + tampilkanStatus(): void

```
public class Mobil
{
    private int kecepatan = 0;
    private boolean kontakOn = false;

    public void nyalakanMesin()
    {
        kontakOn = true;
    }

    public void matikanMesin()
    {
        kontakOn = false;
    }

    public void tambahKecepatan()
    {
        if(kontakOn == true)
        {
            kecepatan += 10;
        }
        else
        {
            System.out.println("Tidak bisa menambah kecepatan, mesin belum menyala!\n");
        }
    }

    public void kurangiKecepatan()
    {
        if(kontakOn == true)
        {
            kecepatan -= 10;
        }
        else
        {
            System.out.println("Tidak bisa mengurangi kecepatan, mesin belum menyala!\n");
        }
    }

    public void tampilkanStatus()
    {
        if(kontakOn == true)
        {
            System.out.println("Kontak ON");
        }
        else
        {
            System.out.println("Kontak OFF");
        }

        System.out.println("Kecepatan: " + kecepatan + "\n");
    }
}
```

Atribut kecepatan dan kontakOn diset *private*

Menerapkan Access Modifier

```
public class TestMobil
{
    public static void main(String[] args)
    {
        Mobil mbl = new Mobil();

        mbl.tampilkanStatus();

        mbl.tambahKecepatan();

        mbl.nyalakanMesin();
        mbl.tampilkanStatus();

        mbl.tambahKecepatan();
        mbl.tampilkanStatus();

        mbl.tambahKecepatan();
        mbl.tampilkanStatus();

        mbl.tambahKecepatan();
        mbl.tampilkanStatus();
    }
}
```

```
D:\MyJava>javac TestMobil.java
D:\MyJava>java TestMobil
Kontak OFF
Kecepatan: 0

Tidak bisa menambah kecepatan, mesin belum menyala!

Kontak ON
Kecepatan: 0

Kontak ON
Kecepatan: 10

Kontak ON
Kecepatan: 20

Kontak ON
Kecepatan: 30
```

Menerapkan Access Modifier

- Dapat kita amati sekarang atribut kecepatan tidak bisa seenaknya diganti.
- Bahkan ketika mencoba menambah kecepatan saat posisi kontak masih OFF, maka akan muncul notifikasi bahwa mesin belum menyala.
- Untuk mendapatkan kecepatan yang diinginkan, maka harus dilakukan secara gradual, yaitu dengan memanggil method `tambahKecepatan()` beberapa kali.
- Hal ini mirip seperti saat kita mengendarai mobil di dunia nyata.

Getter – Setter Method

- Getter – Setter method adalah method yang disediakan untuk mengakses atribut yang diset *private*.
 - Getter digunakan untuk membaca isi atribut, sedangkan Setter untuk mengeset isi atribut.
 - Dengan demikian kita bisa menentukan, apakah suatu atribut dapat dibaca saja, atau hanya diubah saja, atau dapat diubah maupun dibaca.
- 

Menganalisa Skenario

- Misalkan di sebuah sistem informasi perbankan, terdapat class Nasabah.
- Nasabah memiliki atribut nomor rekening, nama dan saldo, dan method setor() dan tarik().
- Saldo pada nasabah tidak boleh diubah sembarangan, melainkan hanya dapat diubah melalui method setor() atau tarik().
- Namun saldo boleh dibaca isi nominalnya.

Menerapkan Getter – Setter Method

Nasabah
- nomorRekening: String - nama: String - saldo: int
+ setNomorRekening(norek: String): void + getNomorRekening(): String + setName(nm: String): void + getName(): String + getSaldo(): int + setor(nominal: int): void + tarik(nominal: int): void

```
public class Nasabah
{
    private String nomorRekening;
    private String nama;
    private int saldo = 0;
```

```
    public void setNomorRekening(String norek)
    {
        nomorRekening = norek;
    }
```

```
    public String getNomorRekening()
    {
        return nomorRekening;
    }
```

```
    public void setName(String nm)
    {
        nama = nm;
    }
```

```
    public String getName()
    {
        return nama;
    }
```

```
    public int getSaldo()
    {
        return saldo;
    }
```

```
    public void setor(int nominal)
    {
        saldo += nominal;
    }
```

```
    public void tarik(int nominal)
    {
        saldo -= nominal;
    }
}
```

Setter – Getter untuk
atribut nomorRekening

Setter – Getter untuk
atribut nama

Getter untuk atribut
saldo

Menerapkan Getter – Setter Method

- Jika diperhatikan class Nasabah diatas, atribut nomorRekening memiliki method setNomorRekening() dan getNomorRekening() sebagai getter-setter nya.
- Atribut nama juga memiliki setName() dan getName().
- Namun, atribut saldo HANYA memiliki getSaldo() saja, karena seperti tujuan awal kita, atribut saldo tidak boleh diubah sembarangan, harus melalui method setor() dan tarik().

Menerapkan Getter – Setter Method

```
public class TestNasabah
{
    public static void main(String[] args)
    {
        Nasabah anton = new Nasabah();

        anton.setNomorRekening("1123123");
        anton.setNama("Anton Kemang");
        anton.setor(10000);

        System.out.println("Saldo " + anton.getNama() + " saat ini: Rp " + anton.getSaldo());

        anton.tarik(5000);

        System.out.println("Saldo " + anton.getNama() + " saat ini: Rp " + anton.getSaldo());
    }
}
```

```
E:\Java>javac TestNasabah.java
```

```
E:\Java>java TestNasabah
Saldo Anton Kemang saat ini: Rp 10000
Saldo Anton Kemang saat ini: Rp 5000
```

Menerapkan Getter – Setter Method

- Dapat dilihat pada hasil percobaan diatas:
 - Untuk mengubah saldo tidak dilakukan secara langsung dengan mengubah atribut saldo, melainkan melalui method `setor()` dan `tarik()`.
 - Untuk menampilkan nama pun harus melalui method `getNama()`, dan untuk menampilkan saldo melalui `getSaldo()`.
- 

Konstruktor

- Konstruktor adalah method yang secara otomatis dieksekusi pertama kali ketika kita membuat object.
- Secara default, java secara otomatis memberikan konstruktor pada tiap class.
- Hal ini dibuktikan pada saat kita membuat objek baru dari sebuah class, maka akan diikuti dengan pemanggilan konstruktornya.
- Misal, membuat objek porsche dari class Mobil:

```
Mobil porsche = new Mobil();
```

- Perhatikan kode **new Mobil()**, artinya adalah memanggil konstruktor dari class Mobil.

Aturan dalam Membuat Konstruktor

- Konstruktor harus memiliki nama yang sama dengan nama class
- Access modifier-nya harus public
- Tidak boleh memiliki tipe data return.
- Contoh penggunaan konstruktor adalah, saat kita ingin memberikan nilai default pada atribut class.
- Cara mendeklarasikan konstruktor adalah sebagai berikut:

```
public <NamaKonstruktor>
{
    // do something
}
```

Contoh Konstruktor

- Misalkan pada class Mobil:
 - Memiliki atribut warna dan jumlahRoda
 - Kita ingin agar ketika dibuat objeknya, maka warna-nya langsung terisi “Putih” dan jumlah roda-nya “4”.

```
public class Mobil
{
    public String warna;
    public int jumlahRoda;

    public Mobil()
    {
        warna = "Putih";
        jumlahRoda = 4;
    }
}
```

```
public class TestMobil
{
    public static void main(String[] args)
    {
        Mobil mbl = new Mobil();

        System.out.println("Warna mobil: " + mbl.warna);
        System.out.println("Jumlah roda: " + mbl.jumlahRoda);
    }
}
```

```
E:\Java\Mobil>java TestMobil
Warna mobil: Putih
Jumlah roda: 4
```

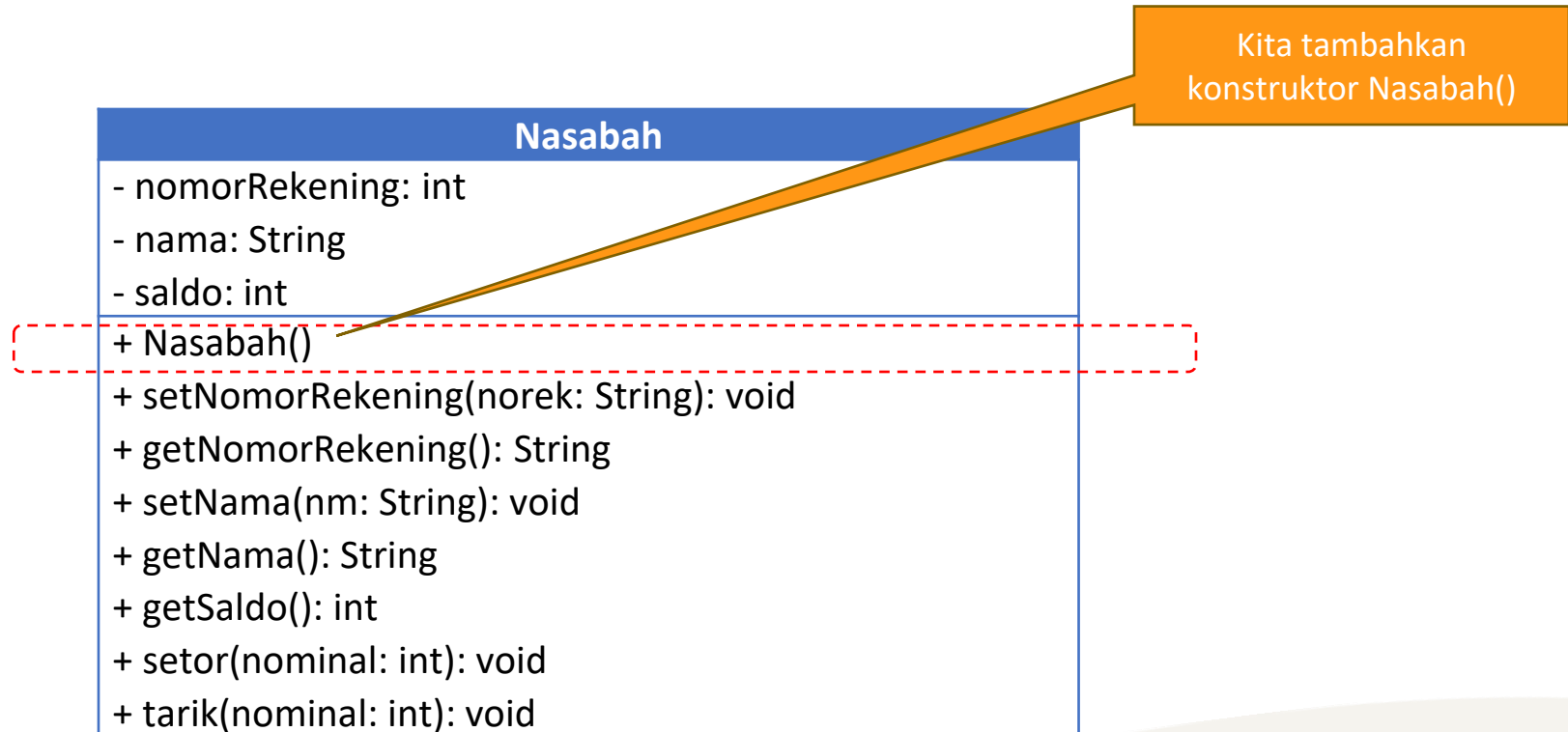
Atribut warna dan jumlahRoda otomatis terisi

Mengeset atribut warna dan jumlahRoda di dalam konstruktor

Menganalisa Skenario

- Misal pada sistem perbankan kita tadi, class Nasabah pada saat pertama kali dibuat objek nya, maka nilai default dari atribut:
 - nomorRekening = "000000"
 - nama = "NO NAME"
 - saldo = 50000
- Bagaimana agar atribut-atribut tersebut terisi nilai defaultnya secara otomatis saat membuat objek?

Menerapkan Konstrukt



```
public class Nasabah
{
    private String nomorRekening;
    private String nama;
    private int saldo = 0;

    public Nasabah()
    {
        nomorRekening = "0000000";
        nama = "NO NAME";
        saldo = 50000;
    }

    public void setNomorRekening(String norek)
    {
        nomorRekening = norek;
    }

    public String getNomorRekening()
    {
        return nomorRekening;
    }

    public void setName(String nm)
    {
        nama = nm;
    }

    public String getName()
    {
        return nama;
    }

    public int getSaldo()
    {
        return saldo;
    }

    public void setor(int nominal)
    {
        saldo += nominal;
    }

    public void tarik(int nominal)
    {
        saldo -= nominal;
    }
}
```

Konstruktor class
Nasabah

Menerapkan Konstruktor

```
public class TestNasabah
{
    public static void main(String[] args)
    {
        Nasabah nas = new Nasabah();

        System.out.println("Nomor rekening: " + nas.getNomorRekening());
        System.out.println("Nama: " + nas.getNama());
        System.out.println("Saldo: " + nas.getSaldo());
    }
}
```

```
D:\MyJava>javac TestNasabah.java
```

```
D:\MyJava>java TestNasabah
```

```
Nomor rekening: 000000
```

```
Nama: NO NAME
```

```
Saldo: 50000
```

Menerapkan Konstruktor

- Amati class TestNasabah diatas.
- Pertama-tama kita membuat objek nas dari class Nasabah.
- Kemudian langsung kita cetak ke layar isi dari atribut-atributnya dengan menggunakan getter methodnya, yaitu `getNomorRekening()`, `getNama()` dan `getSaldo()`.
- Dapat kita lihat bahwa atribut nomor rekening secara otomatis terisi dengan nilai “000000”, atribut nama “NO NAME” dan saldo = 50000.

Konstruktor Yang Menerima Parameter

- Seperti halnya method biasa, konstruktor juga dapat menerima parameter.
- Jika konstruktor menerima parameter, maka pada saat pembuatan object, parameter tersebut juga harus diberi nilai dengan passing parameter.
- Contoh:

```
Mahasiswa mhs = new Mahasiswa("114003005", "Edna", 19, "Malang");
```

- Konstruktor semacam ini biasanya digunakan jika ada suatu kasus dimana pembuatan objek harus disertai dengan pengesetan nilai awal dari beberapa atributnya.

Menganalisa Skenario

- Misal pada saat pembuatan objek dari class Nasabah, harus langsung diisikan atribut nama, nomor rekening dan saldonya.
 - Bagaimana caranya agar dapat mengisi atribut-atribut tersebut langsung melalui konstruktornya?
- 

Menerapkan Konstruktor yang Menerima Parameter

Nasabah
- nomorRekening: int - nama: String - saldo: int
+ Nasabah(norekAwal: String, namaAwal: String, saldoAwal: int)
+ setNomorRekening(norek: String): void + getNomorRekening(): String + setName(nm: String): void + getName(): String + getSaldo(): int + setor(nominal: int): void + tarik(nominal: int): void

```
public class Nasabah
{
    private String nomorRekening;
    private String nama;
    private int saldo = 0;

    public Nasabah(String norekAwal, String namaAwal, int saldoAwal)
    {
        nomorRekening = norekAwal;
        nama = namaAwal;
        saldo = saldoAwal;
    }

    public void setNomorRekening(String norek)
    {
        nomorRekening = norek;
    }

    public String getNomorRekening()
    {
        return nomorRekening;
    }

    public void setName(String nm)
    {
        nama = nm;
    }

    public String getName()
    {
        return nama;
    }

    public int getSaldo()
    {
        return saldo;
    }

    public void setor(int nominal)
    {
        saldo += nominal;
    }

    public void tarik(int nominal)
    {
        saldo -= nominal;
    }
}
```

Konstruktor class
Nasabah yang
menerima parameter

Menerapkan Konstruktor yang Menerima Parameter

```
public class TestNasabah
{
    public static void main(String[] args)
    {
        Nasabah nas = new Nasabah("11556677", "Nikiloe Tesla", 9000000);

        System.out.println("Nomor rekening: " + nas.getNomorRekening());
        System.out.println("Nama: " + nas.getNama());
        System.out.println("Saldo: " + nas.getSaldo());
    }
}
```

Passing parameter pada
konstruktor Nasabah

```
D:\MyJava>javac TestNasabah.java
```

```
D:\MyJava>java TestNasabah
```

```
Nomor rekening: 11556677
```

```
Nama: Nikiloe Tesla
```

```
Saldo: 9000000
```

Pertanyaan

- Apa tujuan utama penggunaan enkapsulasi?
- Berikan contoh kasus sebuah objek di dunia nyata dimana kita harus menerapkan konsep enkapsulasi antara lain:
 - Menggunakan access modifier private pada atribut-atributnya
 - Menggunakan method khusus untuk mengubah nilai atribut-atributnya
- Berikan contoh kasus sebuah objek di dunia nyata dimana kita harus membuat konstruktor pada sebuah class